

White Agent Report: LLM-Based Agent for τ -Bench PM Domain

Alt+Tau: Arnur Sabet, Michael Herrington, Nurzhan Abdrassilov

1 Abstract

Task: We evaluate an LLM-based white agent on tool-calling tasks in a project management (PM) domain within the τ -Bench framework.

Metric: Binary pass¹ reward: 1.0 if the agent’s tool calls match expected actions and required output strings appear in the response, 0.0 otherwise.

Agent Design: Single-call GPT-4o-mini with domain-aware prompting; system prompt contains PM context, tool schemas, and few-shot examples; regex-based JSON parsing with keyword fallback.

Results: The agent achieves 40% pass¹ on 5 PM tasks, compared to 20% for a keyword-matching baseline (2 $\times$ improvement).

Comparison: Ablations show domain context is critical (0% without), while temperature and few-shot examples have no effect; concise prompts (93 tokens) outperform verbose ones (522 tokens).

2 Benchmark

2.1 Related Work

Tool-use benchmarks evaluate agents’ ability to select and invoke APIs correctly. ToolBench [1] tests LLMs on 16K+ real-world APIs. API-Bank [2] provides 314 tool-use dialogues. τ -Bench [3] focuses on *policy-compliant* tool use in customer service, where agents must follow domain rules (e.g., refund limits, status workflows). Prior work reports GPT-4 achieves <50% pass¹ on τ -Bench retail/airline tasks, indicating the difficulty of consistent multi-step tool use under constraints.

2.2 Benchmark Overview

τ -Bench evaluates white agents in simulated environments with:

- **Domain data:** JSON databases (users, orders, tickets, etc.)
- **Tools:** Domain-specific APIs with typed parameters
- **Policies:** Constraints the agent must follow (e.g., valid status transitions)
- **Tasks:** Natural language instructions requiring 1+ tool calls

We extended τ -Bench with a **PM domain**: 5 users, 3 projects, 5 tickets. 9 tools: `list_tickets`,

`create_ticket`, `update_status`, `assign_user`, `update_priority`, `add_comment`, `get_user_details`, `transfer_to_human`, `think`. Policy: status workflow (todo to in_progress to in_review to done), membership constraints.

2.3 Evaluation Protocol

Inputs to white agent: At each step, the white agent receives (1) a natural language instruction describing the task, (2) a list of available tools with JSON schemas specifying parameters and types.

Outputs from white agent: The agent returns a JSON-RPC response containing tool calls, each with a tool name and keyword arguments.

Green agent evaluation: The green agent compares the white agent’s output against ground truth using: (1) *actions_match*: checks if `set(expected_action_names)` equals `set(actual_action_names)`; (2) *outputs_match*: checks if required output strings appear in the response. Reward is 1.0 if both match, else 0.0. Note: set comparison is lenient: it ignores action count, order, and parameter values.

3 White Agent Framework

3.1 Architecture

The white agent is implemented as an async Python service using FastAPI, exposing A2A protocol endpoints. The core `LLMWhiteAgent` class handles task execution via GPT-4o-mini through the LiteLLM abstraction layer.

3.2 Decision-Making Pipeline

1. **Input:** Receive task instruction (string) and available tools (JSON schema array) from green agent
2. **Prompt Construction:** Build system prompt with PM domain context, workflow rules, formatted tool descriptions, and 3 few-shot examples
3. **LLM Invocation:** Call GPT-4o-mini (temperature=0.7, max_tokens=1000) via async LiteLLM wrapper
4. **Output Parsing:** Extract JSON tool calls using regex (handles code blocks and raw JSON)
5. **Fallback:** If parsing fails, use keyword-based rule matching (e.g., “assign” to `assign_user`)

6. **Response:** Return A2A-compliant JSON-RPC response with tool calls

3.3 Modules

Prompt Builder: Constructs domain-aware system prompts including tool descriptions with parameter types. **Tool Formatter:** Converts JSON schemas to readable text for the LLM. **LLM Caller:** Async wrapper around `litellm.completion()`. **JSON Parser:** Regex-based extraction from LLM output. **Fallback Generator:** Rule-based keywordtoaction mapping for robustness.

3.4 Reasoning Techniques

Tool-augmented reasoning with complete parameter specifications; domain-specific policy rules embedded in system prompt; few-shot examples showing expected JSON format; graceful degradation via fallback mechanism.

4 Experiments

4.1 Performance vs Baselines (Q8.1)

Agent	pass ¹	Tasks
Keyword baseline	20%	1/5
GPT-4o-mini (our agent)	40%	2/5

The LLM agent doubles baseline performance. The baseline maps keywords to single actions; the LLM handles multi-action tasks (e.g., creating two tickets).

4.2 Generalizability (Q8.2)

The agent generalizes via: (1) tool-agnostic architecture: works with any tool schema provided at runtime; (2) no fine-tuning: uses general-purpose GPT-4o-mini; (3) domain knowledge via prompt only. Limitation: tested only on PM domain; other τ -Bench domains require prompt adaptation.

4.3 Reasoning Quality (Q8.3)

Success (Task 1): “Create two tickets with different priorities” to LLM correctly generates two `create_ticket` calls with proper titles and priorities.

Failure (Task 0): “Assign all high-priority tickets” to LLM outputs `list_tickets` instead of `assign_user`. *Analysis:* The LLM tries to query before acting, even when ticket IDs are provided. This “query-before-act” pattern is a systematic failure mode.

Additional failure modes: (1) *tool invention*: LLM generates non-existent tools like `update_ticket`; (2) *action omission*: LLM misses steps in multi-action tasks.

4.4 Efficiency (Q8.4)

Metric	Value
LLM calls/task	1
Prompt tokens (avg)	93
Completion tokens (avg)	54
Latency	1–2 sec
Cost/task	\$0.0002

4.5 Bias/Overfitting/Contamination (Q8.5)

No fine-tuning: uses off-the-shelf GPT-4o-mini. No data leakage: PM tasks created by team, not in training data. Neutral data: generic user names (Alice, Bob, Carol). Reproducible: consistent results across 3 runs.

4.6 Impact/Reusability/Documentation (Q8.6)

Modular design: `LLMWhiteAgent` class works with any LLM via LiteLLM. A2A Protocol: standard interface enables integration with any evaluator. Documentation: README with setup instructions. Runnable: `launcher.py` runs full assessment; `test_simple.py` for verification.

References: [1] Qin et al., ToolLLM, 2023. [2] Li et al., API-Bank, 2023. [3] Yao et al., τ -Bench, 2024.